



[LO5] Logique

0. Organisation du cours

Sylvain SCHMITZ



Licence 3e année
UFR Informatique



ORGANISATION

- la page du cours

https://www.irif.fr/~schmitz/teach/2025_1o5/

contient les notes de cours, les feuilles de TD, les dates d'examen et les consignes





MATÉRIEL PÉDAGOGIQUE

INTRODUCTION À LA LOGIQUE INFORMATIQUE

SYLVAIN SCHMITZ

Université de Paris, France

CONTENU DES NOTES

Ce cours fournit une introduction à la logique informatique. L'accent est mis sur l'utilisation de la logique pour modéliser des problèmes, et les résoudre ensuite à l'aide de solveurs comme les solveurs SAT et les solveurs SMT, dont on voit aussi les principes de fonctionnement. Le cours revisite la logique propositionnelle vue en cours d'« outils logiques » (IL4), et introduit la logique du premier ordre. Le programme général est :

- *Logique propositionnelle : syntaxe et sémantique. Conséquences et équivalences logiques. Formes normales, forme clause. Modélisation, solveurs SAT, recherche de modèle, algorithme DPLL. Recherche de preuve, calcul des séquents propositionnel.*
- *Logique du premier ordre : syntaxe et sémantique. Formes normales, skolemisation. Théories logiques, interprétations normales, élimination des quantificateurs. Modélisation, solveurs SMT. Recherche de preuve, calcul des séquents du premier ordre.*

Les sous-ajonctions dont le titre est précédé d'une astérisque « * » et grisé(e) dans le texte apportent des compléments qui ne seront pas traités en cours.

Partie 1. Introduction	5
1. Contexte et motivations	5
1.1. Logique philosophique	5
1.2. Logique mathématique	5
1.3. Logique informatique	6
1.3.1. Circuits logiques	6
1.3.2. Complexité algorithmique	6
1.3.3. Problèmes « combinatoires »	7
1.3.4. Programmation logique	7
1.3.5. Bases de données	7
1.3.6. Vérification de programmes	8
1.4. Des nombreuses logiques	8
Partie 2. Logique classique propositionnelle	9
2. Syntaxe	9
2.1. Arbres de syntaxe abstraite	9
2.1.1. Représentation en Java	10
2.1.2. Représentation en OCaml	11
2.2. Syntaxe concrète	12



les notes de cours



MODALITÉS DE CONTRÔLE DES CONNAISSANCES

- ▶ QCM_1 , QCM_2 , QCM_3 : trois QCM
- ▶ MP_1 : un mini-projet en binômes
- ▶ PA : partiel
- ▶ EX : examen

NOTE DE PREMIÈRE SESSION

$$0,1 \cdot QCM_1 + 0,1 \cdot MP_1 + 0,2 \cdot PA + 0,1 \cdot QCM_2 + 0,1 \cdot QCM_3 + 0,4 \cdot EX$$



CONTENU DU COURS

- ▶ Partie 1. Introduction
- ▶ Partie 2. Logique propositionnelle
- ▶ Partie 3. Logique du premier ordre

OBJECTIFS DU COURS

- ▶ **modéliser** à l'aide de formules logiques
(encodage de problèmes combinatoires, formats DIMACS et SMT-LIB, axiomatisations, ...)
- ▶ **manipuler** des formules logiques
(forme normale négative, forme clausale, skolemisation, ...)
- ▶ **raisonner** sur des formules logiques
(algorithme DPLL, calcul des séquents, ...)



[LO5] Logique

1. Contexte et motivations

Sylvain SCHMITZ



Licence 3e année
UFR Informatique



WIKIPEDIA

La logique, du grec λογική / logikê, est un terme dérivé de λογος / lógos — signifiant à la fois « raison », « langage » et « raisonnement » – est, dans une première approche, l'étude des règles formelles que doit respecter toute argumentation correcte.

Elle est depuis l'Antiquité l'une des grandes disciplines de la philosophie [...]. En outre, on a assisté depuis le XIXe siècle au développement fulgurant d'une approche mathématique de la logique. Sa convergence opérée avec l'informatique depuis la fin du XXe siècle lui a donné un regain de vitalité.

LOGIQUE PHILOSOPHIQUE

quels sont les raisonnements concluants?

- ▶ « Tous les humains sont mortels, or SOCRATE est humain, donc SOCRATE est mortel »
- ▶ « Toutes les souris sont vertes, or YODA est une souris, donc YODA est vert »
- ▶ « Tous les humains sont mortels, or SOCRATE est mortel, donc SOCRATE est humain »

LOGIQUE PHILOSOPHIQUE

quels sont les raisonnements concluants?

- ▶ « Tous les humains sont mortels, or SOCRATE est humain, donc SOCRATE est mortel »

formule logique valide :

$$\left((\forall x. H(x) \Rightarrow M(x)) \wedge H(s) \right) \Rightarrow M(s)$$

LOGIQUE PHILOSOPHIQUE

quels sont les raisonnements concluants?

- ▶ « Tous les humains sont mortels, or SOCRATE est humain, donc SOCRATE est mortel »

formule logique valide :

$$\left((\forall x. H(x) \Rightarrow M(x)) \wedge H(s) \right) \Rightarrow M(s)$$

LOGIQUE MATHÉMATIQUE

- ▶ depuis le XIXe siècle : formalisation des mathématiques
- ▶ « crise des fondements »

PARADOXES

Un barbier rase tous les hommes qui ne se rasent pas eux-mêmes (et seulement ceux-là); est-ce que ce barbier se rase lui-même?

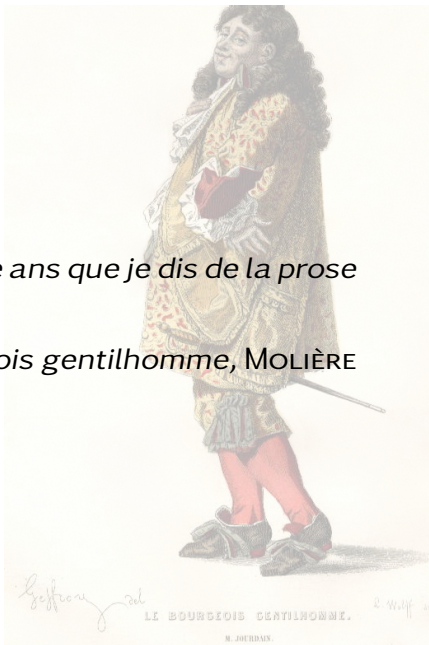
- ▶ développement de la logique



LOGIQUE PARTOUT

Par ma foi! Il y a plus de quarante ans que je dis de la prose sans que j'en susse rien.

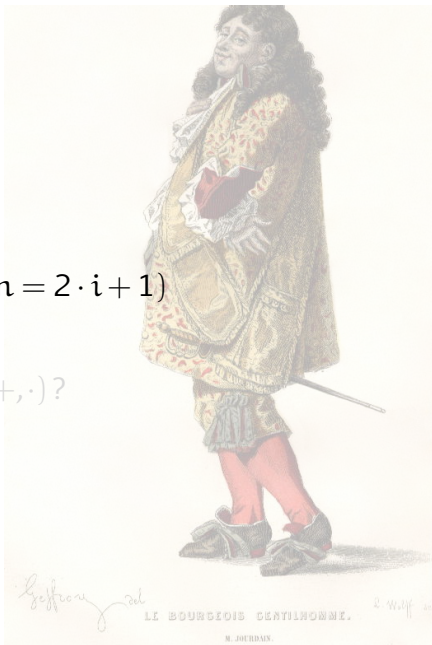
Le bourgeois gentilhomme, MOLIERE



ÉNONCÉS MATHÉMATIQUES

$$\forall n. \exists i. (n = 2 \cdot i) \vee (n = 2 \cdot i + 1)$$

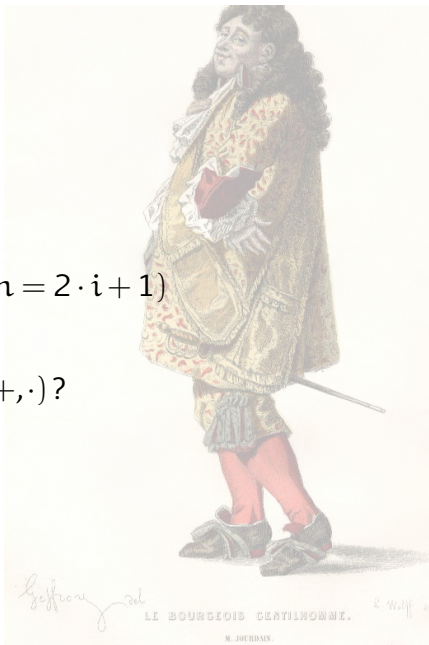
- la formule est-elle vraie sur $(\mathbb{N}, +, \cdot)$?



ÉNONCÉS MATHÉMATIQUES

$$\forall n. \exists i. (n = 2 \cdot i) \vee (n = 2 \cdot i + 1)$$

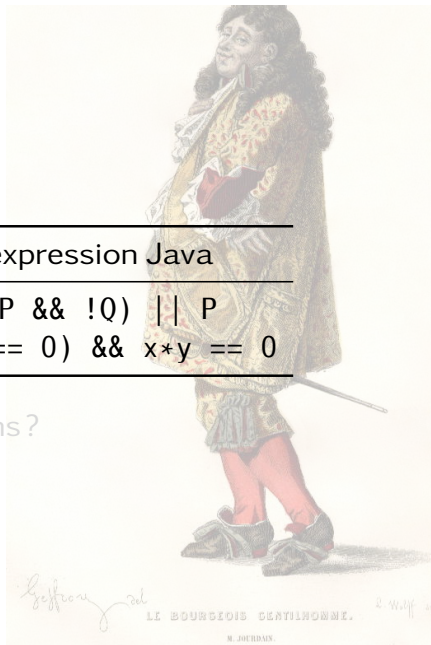
- la formule est-elle vraie sur $(\mathbb{N}, +, \cdot)$?



EXPRESSIONS BOOLÉENNES

formule logique	expression Java
$(P \wedge \neg Q) \vee P$	<code>(P && !Q) P</code>
$\neg(x = 0) \wedge x \cdot y = 0$	<code>!(x == 0) && x*y == 0</code>

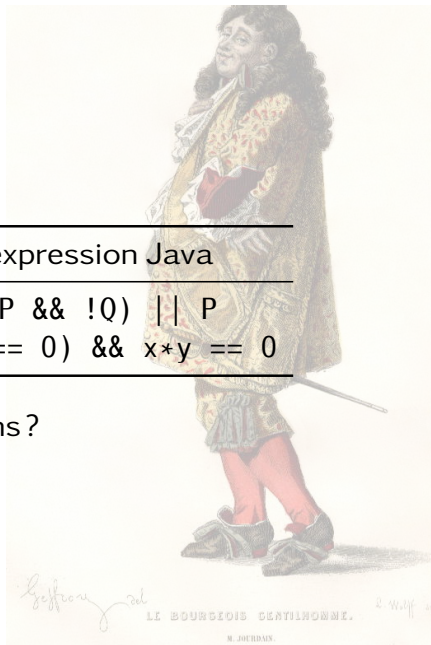
► peut-on simplifier ces expressions ?



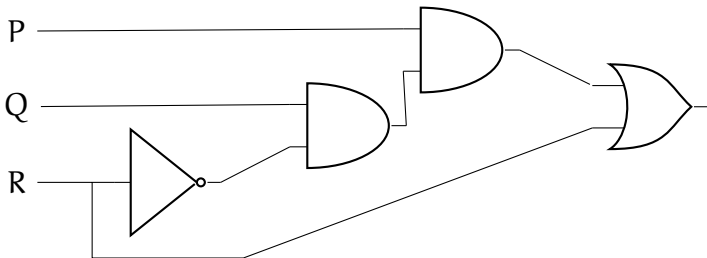
EXPRESSIONS BOOLÉENNES

formule logique	expression Java
$(P \wedge \neg Q) \vee P$	<code>(P && !Q) P</code>
$\neg(x = 0) \wedge x \cdot y = 0$	<code>!(x == 0) && x*y == 0</code>

- peut-on simplifier ces expressions?



CIRCUITS BOOLÉENS

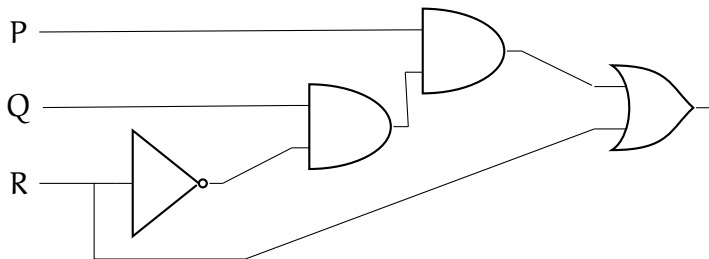


réalisation matérielle de la formule propositionnelle

$$(P \wedge (Q \wedge \neg R)) \vee R$$

- ▶ comment construire un circuit ?
- ▶ peut-on minimiser un circuit ?
- ▶ cours « circuits et architecture » en M1

CIRCUITS BOOLÉENS

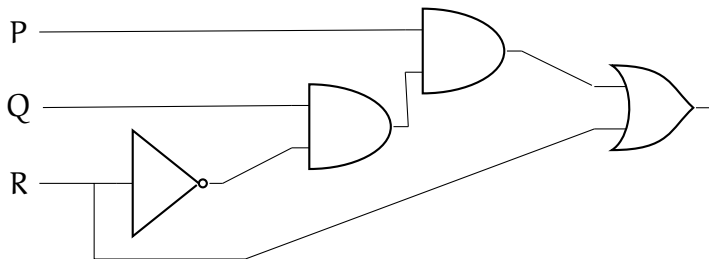


réalisation matérielle de la formule propositionnelle

$$(P \wedge (Q \wedge \neg R)) \vee R$$

- ▶ comment construire un circuit ?
- ▶ peut-on minimiser un circuit ?
- ▶ cours « circuits et architecture » en M1

CIRCUITS BOOLÉENS



réalisation matérielle de la formule propositionnelle

$$(P \wedge (Q \wedge \neg R)) \vee R$$

- ▶ comment construire un circuit ?
- ▶ peut-on minimiser un circuit ?
- ▶ cours « **circuits et architecture** » en M1

PROGRAMMER $\approx$ PROUVER



L'isomorphisme de CURRY-HOWARD au cœur du fonctionnement des assistants de preuve et de la théorie des types.

- cours « **preuves assistées par ordinateur** » en M1



PROGRAMMER $\approx$ PROUVER



L'isomorphisme de CURRY-HOWARD au cœur du fonctionnement des assistants de preuve et de la théorie des types.

- cours « **preuves assistées par ordinateur** » en M1

COMPLEXITÉ ALGORITHMIQUE

problèmes logiques emblématiques des problèmes rencontrés en informatique

problème logique	complexité
2SAT	NL-complet
HornSAT	P-complet
SAT	NP-complet
QBF	PSPACE-complet

- ▶ les problèmes informatiques s'**encodent** comme des problèmes logiques
- ▶ cours « **calculabilité et complexité** » en M1

COMPLEXITÉ ALGORITHMIQUE

problèmes logiques emblématiques des problèmes rencontrés en informatique

problème logique	complexité
2SAT	NL-complet
HornSAT	P-complet
SAT	NP-complet
QBF	PSPACE-complet

- ▶ les problèmes informatiques s'**encodent** comme des problèmes logiques
- ▶ cours « **calculabilité et complexité** » en M1

COMPLEXITÉ ALGORITHMIQUE

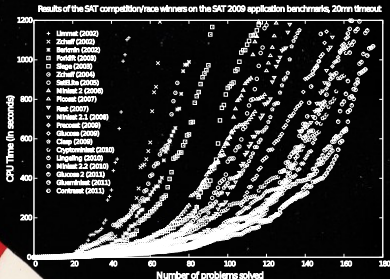
problèmes logiques emblématiques des problèmes rencontrés en informatique

problème logique	complexité
2SAT	NL-complet
HornSAT	P-complet
SAT	NP-complet
QBF	PSPACE-complet

- ▶ les problèmes informatiques s'**encodent** comme des problèmes logiques
- ▶ cours « **calculabilité et complexité** » en M1

LA RÉVOLUTION SAT

- ▶ solveurs pour la satisfiabilité de formules propositionnelles
- ▶ extrêmement performants



RÉSOLUTION DE CONTRAINTES

Exemple : sudoku

démo : <https://blag.cedeela.fr/sudoku-msat/>

					3		8	5
		1		2				
			5		7			
		4				1		
	9							
5							7	3
		2		1				
				4				9

- ▶ aussi : faire un emploi du temps, planifier des tâches, trouver une installation optimale de paquets linux, etc.
- ▶ cours « programmation logique et par contraintes » en M1



RÉSOLUTION DE CONTRAINTES

Exemple : sudoku

démo : <https://blag.cedeela.fr/sudoku-msat/>

9	8	7	6	5	4	3	2	1
2	4	6	1	7	3	9	8	5
3	5	1	9	2	8	7	4	6
1	2	8	5	3	7	6	9	4
6	3	4	8	9	2	1	5	7
7	9	5	4	6	1	8	3	2
5	1	9	2	8	6	4	7	3
4	7	2	3	1	9	5	6	8
8	6	3	7	4	5	2	1	9

- ▶ aussi : faire un emploi du temps, planifier des tâches, trouver une installation optimale de paquets linux, etc.
- ▶ cours « programmation logique et par contraintes » en M1



RÉSOLUTION DE CONTRAINTES

Exemple : sudoku

démo : <https://blag.cedeela.fr/sudoku-msat/>

9	8	7	6	5	4	3	2	1
2	4	6	1	7	3	9	8	5
3	5	1	9	2	8	7	4	6
1	2	8	5	3	7	6	9	4
6	3	4	8	9	2	1	5	7
7	9	5	4	6	1	8	3	2
5	1	9	2	8	6	4	7	3
4	7	2	3	1	9	5	6	8
8	6	3	7	4	5	2	1	9

- ▶ aussi : faire un emploi du temps, planifier des tâches, trouver une installation optimale de paquets linux, etc.
- ▶ cours « programmation logique et par contraintes » en M1

RÉSOLUTION DE CONTRAINTES

Exemple : sudoku

démo : <https://blag.cedeela.fr/sudoku-msat/>

9	8	7	6	5	4	3	2	1
2	4	6	1	7	3	9	8	5
3	5	1	9	2	8	7	4	6
1	2	8	5	3	7	6	9	4
6	3	4	8	9	2	1	5	7
7	9	5	4	6	1	8	3	2
5	1	9	2	8	6	4	7	3
4	7	2	3	1	9	5	6	8
8	6	3	7	4	5	2	1	9

- ▶ aussi : faire un emploi du temps, planifier des tâches, trouver une installation optimale de paquets linux, etc.
- ▶ cours « programmation logique et par contraintes » en M1

BASE DE DONNÉES

- une requête SQL

```
SELECT Vols.depart
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom
WHERE Aeroports.pays = 'FR'
```

Vols	
depart	arrivee
'ATL'	'CDG'
'LAX'	'ATL'
'ORY'	'FRA'
'CDG'	'PEK'
'AMS'	'ORY'
'AMS'	'ATL'
'PEK'	'LAX'

Aeroports	
nom	pays
'ATL'	'US'
'PEK'	'CN'
'LAX'	'US'
'CDG'	'FR'
'AMS'	'NL'
'FRA'	'DE'
'ORY'	'FR'

BASE DE DONNÉES

- une requête SQL

SELECT Vols.depart

FROM Vols **JOIN** Aeroports **ON** Vols.arrivee = Aeroports.nom

WHERE Aeroports.pays = 'FR'

Vols ⋈ _{arrivee=nom} Aeroports		
depart	arrivee	pays
'ATL'	'CDG'	'FR'
'LAX'	'ATL'	'US'
'ORY'	'FRA'	'DE'
'CDG'	'PEK'	'CN'
'AMS'	'ORY'	'FR'
'AMS'	'ATL'	'US'
'PEK'	'LAX'	'US'

BASE DE DONNÉES

- une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

$\sigma_{\text{pays}='FR'}(\text{Vols} \bowtie_{\text{arrivee}=\text{nom}} \text{Aeroports})$		
depart	arrivee	pays
'ATL'	'CDG'	'FR'
'AMS'	'ORY'	'FR'

BASE DE DONNÉES

- une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

$$\pi_{\text{depart}}(\sigma_{\text{pays}='FR'}(\text{Vols} \bowtie_{\text{arrivee}=\text{nom}} \text{Aeroports}))$$

'ATL'

'AMS'

BASE DE DONNÉES

- ▶ une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

- ▶ ... n'est jamais qu'une formule logique

$$\exists arrivee \exists nom \exists pays . Vols(depart, arrivee) \\ \wedge Aeroports(nom, pays) \wedge arrivee = nom \wedge pays = 'FR'$$

- ▶ comment évaluer une requête ?
- ▶ peut-on optimiser les requêtes ?
- ▶ cours de « bases de données » en L3 et de « bases de données avancées » en M1

BASE DE DONNÉES

- une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

- ... n'est jamais qu'une formule logique

$$\exists arrivee \exists nom \exists pays . Vols(depart, arrivee) \\ \wedge Aeroports(nom, pays) \wedge arrivee = nom \wedge pays = 'FR'$$

- comment évaluer une requête?

- peut-on optimiser les requêtes?

- cours de « bases de données » en L3 et de « bases de données avancées » en M1

BASE DE DONNÉES

- ▶ une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

- ▶ ... n'est jamais qu'une formule logique

$$\exists arrivee \exists nom \exists pays . Vols(depart, arrivee) \\ \wedge Aeroports(nom, pays) \wedge arrivee = nom \wedge pays = 'FR'$$

- ▶ comment évaluer une requête?
- ▶ peut-on optimiser les requêtes?
- ▶ cours de « bases de données » en L3 et de « bases de données avancées » en M1

BASE DE DONNÉES

- ▶ une requête SQL

```
SELECT Vols.depart  
FROM Vols JOIN Aeroports ON Vols.arrivee = Aeroports.nom  
WHERE Aeroports.pays = 'FR'
```

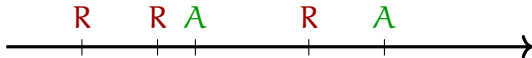
- ▶ ... n'est jamais qu'une formule logique

$$\exists arrivee \exists nom \exists pays . Vols(depart, arrivee) \\ \wedge Aeroports(nom, pays) \wedge arrivee = nom \wedge pays = 'FR'$$

- ▶ comment évaluer une requête?
- ▶ peut-on optimiser les requêtes?
- ▶ cours de « bases de données » en L3 et de « bases de données avancées » en M1

SPÉCIFICATION DE PROGRAMMES

- ▶ une exécution d'un système comprenant un client et un serveur



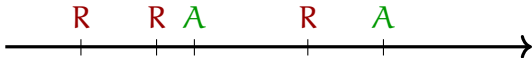
- ▶ une spécification logique

$$\forall t. \text{R}(t) \Rightarrow \exists t'. t \leq t' \wedge \text{A}(t')$$

- ▶ comment montrer que toutes les exécutions du système satisfont la spécification ?
- ▶ cours « modélisation et spécification »

SPÉCIFICATION DE PROGRAMMES

- ▶ une exécution d'un système comprenant un client et un serveur



- ▶ une spécification logique

$$\forall t. \text{R}(t) \Rightarrow \exists t'. t \leq t' \wedge \text{A}(t')$$

- ▶ comment montrer que toutes les exécutions du système satisfont la spécification ?
- ▶ cours « modélisation et spécification »

SPÉCIFICATION DE PROGRAMMES

- ▶ une exécution d'un système comprenant un client et un serveur



- ▶ une spécification logique

$$\forall t. \text{R}(t) \Rightarrow \exists t'. t \leq t' \wedge \text{A}(t')$$

- ▶ comment montrer que toutes les exécutions du système satisfont la spécification ?
- ▶ cours « modélisation et spécification »

```
int i = 0;
int r = 1;
while ( i != n ) {
    i++;
    r *= i;
}
```

PREUVES DE PROGRAMMES

```
int i = 0;
int r = 1;
while ( i != n ) {
    i++;
    r *= i;
}
```

- comment montrer que ce code calcule « $n!$ »?
- preuve en logique de HoARE

PREUVES DE PROGRAMMES

```
int i = 0;
int r = 1;
while ( i != n ) {
    i++;
    r *= i;
}
```

- comment montrer que ce code calcule $\ll n! \gg$?
- preuve en logique de HOARE

$$\frac{\frac{\frac{\overline{\{r(i+1) = (i+1)!\} \ i++ \ \{ri = i!\}}_{\text{csq}}}{\{r = i!\} \ i++ \ \{ri = i!\}}_{\text{seq}}}{\frac{\overline{\{i = 0\} \ \text{int} \ r=1 \ \{r = 1 \wedge i = 0\}}_{\text{init}}}{\{i = 0\} \ \text{int} \ r=1 \ \{r = i!\}}_{\text{csq}} \quad \frac{\frac{\overline{\{r = i!\} \ i++ \ ; r*=i \ \{r = i!\}}_{\text{csq}}}{\{r = i!\} \wedge i \neq n \ i++ \ ; r*=i \ \{r = i!\}}_{\text{while}}}{\{r = i!\} \ \text{while} \ (i \neq n) \ \{i++ \ ; r*=i \ \{r = i!\} \wedge i = n\}}_{\text{while}}}$$

PREUVES DE PROGRAMMES

```
int i = 0;
int r = 1;
while ( i != n ) {
    i++;
    r *= i;
}
```

- comment montrer que ce code calcule $\ll n! \gg$?
- preuve en logique de HoARE

$$\frac{\frac{\frac{\overline{\{r(i+1) = (i+1)!\} \ i++ \ \{ri = i!\}}_{\text{csq}}^{\text{aff}}}{\overline{\{r = i!\} \ i++ \ \{ri = i!\}}}}{\overline{\{r = i!\} \ i++ \ ; r* = i \ \{r = i!\}}}_{\text{seq}}^{\text{aff}}}{\overline{\{i = 0\} \ \text{int} \ r = 1 \ \{r = 1 \wedge i = 0\}}}_{\text{csq}}^{\text{init}}}{\overline{\{i = 0\} \ \text{int} \ r = 1 \ \{r = i!\}}}_{\text{csq}}^{\text{init}}}{\overline{\{r = i!\} \ \text{while}(i != n) \ \{i++ ; r* = i\} \ \{r = i! \wedge i = n\}}}_{\text{while}}^{\text{csq}}}{\overline{\{i = 0\} \ \text{int} \ r = 1 ; \text{while}(i != n) \ \{i++ ; r* = i\} \ \{r = i! \wedge i = n\}}}_{\text{seq}}^{\text{init}}}_{\overline{\{\top\} \ \text{int} \ i = 0 \ \{i = 0\}}}_{\text{init}}^{\text{init}}$$

- cours « méthodes formelles de vérification »



DES LOGIQUES

- ▶ logique classique propositionnelle vue en cours d'« outils logiques » en L2
- ▶ on verra aussi la logique classique du premier ordre dans la deuxième moitié du cours
- ▶ quantités de logiques!
modale, temporelle, d'ordre supérieur, intuitioniste, linéaire, ...
- ▶ répondre à des besoins de modélisation et de raisonnement formels



DES LOGIQUES

- ▶ logique classique propositionnelle vue en cours d'« outils logiques » en L2
- ▶ on verra aussi la logique classique du premier ordre dans la deuxième moitié du cours
- ▶ quantités de logiques!
modale, temporelle, d'ordre supérieur, intuitioniste, linéaire, ...
- ▶ répondre à des besoins de modélisation et de raisonnement formels

DES LOGIQUES

- ▶ logique classique propositionnelle vue en cours d'« outils logiques » en L2
- ▶ on verra aussi la logique classique du premier ordre dans la deuxième moitié du cours
- ▶ quantités de logiques!
modale, temporelle, d'ordre supérieur, intuitioniste, linéaire, ...
- ▶ répondre à des besoins de modélisation et de raisonnement formels

DES LOGIQUES

- ▶ logique classique propositionnelle vue en cours d'« outils logiques » en L2
- ▶ on verra aussi la logique classique du premier ordre dans la deuxième moitié du cours
- ▶ quantités de logiques!
modale, temporelle, d'ordre supérieur, intuitioniste, linéaire, ...
- ▶ répondre à des besoins de modélisation et de raisonnement formels